



GeoTracker — Presentation Guide

(Written for a complete beginner — presentation is TOMORROW, don't panic!)



What Is This Project? (Say this when they ask)

GeoTracker is an employee attendance management system. Instead of signing a paper register, employees check in and check out using their phone or laptop. The system uses **GPS location** to make sure the employee is physically present at the office before they can check in. The admin can see who came, when they came, how long they worked, and can approve or reject leave requests — all from one dashboard.



What Tech Is Used? (Just memorize this list)

What	Technology
UI (what you see)	React (a JavaScript library)
Language	TypeScript (JavaScript with rules)
Styling	Tailwind CSS (ready-made design classes)
Maps	Leaflet (open-source map library)
Build tool	Vite (runs the app locally super fast)
Data (demo)	JSON files (no real database needed for demo)



HOW TO RUN THE PROJECT (Step by Step)

⚠ Do this BEFORE your presentation! Run the app, keep the terminal open, and just switch to the browser during presentation.

Step 1 — Open a Terminal

Open your terminal in the project folder. The folder is:

```
/home/thor/testGeo/GeoTracker-Demo
```

Or just right-click inside the folder → "Open Terminal Here"

Step 2 — Install Dependencies (only needed once)

```
npm install
```

This downloads all the libraries the project needs. Wait for it to finish. You'll see a `node_modules` folder appear.

Step 3 — Start the App

```
npm run dev
```

You should see something like:

```
VITE v6.x.x  ready in 300ms  
  
→ Local:   http://localhost:3000/
```

Step 4 — Open the Browser

Go to: <http://localhost:3000>

🎉 The app is now running!

✅ To Stop the App

Press `Ctrl + C` in the terminal.

🔑 Login Credentials (for the demo)

Role	Username	Password
Admin	admin	pass
Employee	Yash	pass
Employee	Lalit	pass
Employee	Tanisha	pass
Employee	Kartik	pass

💡 **Tip for presentation:** Log in as **admin** first to show the dashboard, then log out and log in as an employee like **Yash** to show the employee view.

📋 FEATURE WALKTHROUGH (What to show & what to say)

🔑 1. Login Screen

- **What to show:** Open <http://localhost:3000>, type `admin` and `pass`, click Login.
- **What to say:** "This is the login page. We have role-based access — admin sees a management dashboard, employees see their own attendance page."

🏠 2. Admin Dashboard — Leave Requests Tab

- **What to show:** After login as admin, the "Leave Requests" tab opens first. You'll see pending requests from Tanisha and Kartik. Click **Approve** on one.
- **What to say:** "Admin can see all pending leave requests here. One click to approve or reject."

📅 3. Admin Dashboard — Attendance Calendar Tab

- **What to show:** Click "Attendance Calendar" in the sidebar. Select "Yash" from the dropdown. A calendar will appear showing which days he was present.
 - **What to say:** *"The admin can view any employee's attendance month by month. The summary shows total days present, total hours worked, and leave days taken."*
-

4. Admin Dashboard — Working Hours Tab

- **What to show:** Click "Working Hours" in the sidebar. A table shows all employees, days present, total hours, and average hours per day.
 - **What to say:** *"This is useful for payroll calculations. You can see exactly how many hours each employee worked."*
-

5. Admin Dashboard — Employees Tab

- **What to show:** Click "Employees". You see a list of all employees. Click "Add Employee" to show the form.
 - **What to say:** *"Admin can add or remove employees directly from here."*
-

6. Admin Dashboard — Geofence Tab

- **What to show:** Click "Geofence". Click the pencil icon on any employee to show the editor.
 - **What to say:** *"Geofencing means we draw a virtual boundary around the office. If an employee tries to check in from outside this circle, the system won't allow it. Admin can set the radius per employee."*
-

7. Employee Dashboard

- **What to show:** Log out from admin. Log in as `Yash` / `pass`. The employee dashboard opens.
 - **What to say:** *"This is what an employee sees. They can see their check-in status, a map showing their current location and the office geofence, and their leave history."*
-

LIKELY QUESTIONS & ANSWERS

Basic Questions

Q: What problem does this solve?

Traditional attendance systems require physical registers or punch cards. This app makes attendance digital, location-verified, and manageable from anywhere.

Q: Is this a real production app?

This is a demo version. It runs with local JSON data files instead of a real database. The code is structured to connect to a real backend — you just swap the data files for actual API calls.

Q: What is GPS / Geofencing?

GPS gives us the employee's latitude and longitude (their exact position on earth). Geofencing means we define a circle around the office. If the employee's location is inside the circle, they're allowed to check in. If outside, the system blocks them.

Q: What happens if someone fakes their GPS location?

GPS spoofing is a real challenge. In production, additional security layers like device verification or IP checks can be added. For this demo, we trust the browser's location.

● Technical Questions

Q: What is React?

React is a JavaScript library by Facebook for building user interfaces. Instead of reloading the whole page when something changes, React updates only the part that changed. This makes apps feel fast and smooth.

Q: What is TypeScript?

TypeScript is JavaScript with extra rules. It tells developers "this function expects a number, not a string." This catches bugs before the code even runs. Very popular in large projects.

Q: What is Vite?

Vite is a build tool — it makes the app run on your computer during development. It's much faster than older tools. `npm run dev` starts the Vite server.

Q: What is Tailwind CSS?

Tailwind is a CSS framework. Instead of writing custom CSS, you add small class names directly in HTML like `text-blue-500 text-lg`. Makes styling very fast.

Q: What is Leaflet / the map library?

Leaflet is a free, open-source library for interactive maps. It uses OpenStreetMap (like Google Maps but free). We use it to show the employee's live location and the office geofence circle.

Q: Why are there JSON files? Where is the database?

For the demo, we use simple JSON files (`users.json` , `attendance.json` , `leaves.json`) in the `data/` folder. This means the demo works without setting up a database server. In a real deployment, these would be replaced by a database like PostgreSQL or MongoDB.

Q: What is an API?

API (Application Programming Interface) is how the frontend talks to the backend. The frontend sends a request like "give me all users" and the backend sends back the data. In this demo, that's simulated using local JSON files.

Q: What is localStorage?

localStorage is a small storage space in the browser. We use it to remember the logged-in user even if you refresh the page — like "remember me" functionality.

Q: What is role-based access control?

*Different users get different permissions based on their role. An **Admin** can see all employees, approve leaves, set geofences. An **Employee** can only see their own attendance and apply for leave.*

● Feature Questions

Q: How does check-in work?

When the employee clicks "Check In", the browser asks for their GPS location. The app checks if that location is inside the geofence circle. If yes, a check-in record is created with the timestamp. If no, it's denied.

Q: What is the default geofence?

The default office location is coordinates `21.0125°N, 75.5026°E` (Maharashtra, India) with a radius of **1 kilometre**. Each employee can have their own custom geofence set by the admin.

Q: Can the admin see where an employee is right now (live tracking)?

No. The system only captures location at check-in time. It does not continuously track employees. This is better for privacy.

Q: What types of leaves are supported?

Three types: **Casual Leave**, **Medical/Sick Leave**, and **Other**.

Q: Can employees see if their leave was approved?

Yes. In the employee dashboard, they can see all their leave requests and their current status — Pending, Approved, or Rejected.

Q: What does the attendance calendar show?

A monthly calendar where each day is colour-coded — present days, leave days, and empty days (weekends/no record). The summary shows total present days, working hours, and leave days.

Q: Can you export data?

Yes. There is an export feature that allows downloading attendance data as a **CSV file**, which opens in Excel.

● Advanced Questions

Q: How would you scale this for 1000 employees?

Replace JSON files with a proper database (PostgreSQL or MongoDB). Use a backend server (Node.js/Express or Spring Boot). Add caching with Redis. Use cloud hosting like AWS or GCP. The frontend code stays almost the same.

Q: What security concerns does this app have?

1. Passwords in JSON files are plain text (production needs bcrypt hashing). 2. The demo token is not a real JWT. 3. GPS spoofing risk. 4. In production, all data would be sent over HTTPS.

Q: Why use React instead of plain HTML/CSS/JS?

React makes it much easier to build complex UIs with interactive parts (modals, tabs, live updates). Plain HTML/JS becomes hard to manage as the app grows.

Q: How is the project structured?

Clean separation of concerns:

- `components/` — All UI screens and modals
 - `services/` — All API/data fetching logic
 - `data/` — Demo JSON data files
 - `types.ts` — Data shape definitions
 - `App.tsx` — Main file that ties everything together
-

PRESENTATION TIPS

1. **Open the app before presenting.** Have it running at localhost:3000 already.
2. **Log in as admin first.** Show the Leave Requests tab → Attendance Calendar → Geofence tab.
3. **Then show employee view.** Log out and log in as Yash.
4. **If something breaks,** stay calm and say: *"This is a demo environment — in production this would connect to a real backend."*
5. **Have this cheat sheet open** on your phone or a second screen.
6. **Speak slowly.** You know this project better than anyone in the room.

Project File Structure (Quick Reference)

```
GeoTracker-Demo/
|
├─ index.html          ← The single HTML page the browser loads
├─ index.tsx           ← Entry point – starts React
├─ App.tsx             ← Main app logic (login, routing between pages)
├─ types.ts            ← Defines data shapes (User, Leave, etc.)
├─ constants.ts        ← Default geofence location
├─ index.css           ← Base CSS styles
|
├─ components/
|   ├── LoginScreen.tsx ← Login page
|   ├── AdminDashboard.tsx ← Admin's main page (tabs)
|   ├── EmployeeDashboard.tsx ← Employee's main page
|   ├── AttendanceCalendar.tsx ← Monthly calendar UI
|   ├── LeaveRequestModal.tsx ← Popup to apply for leave
|   ├── LeaveHistoryModal.tsx ← Popup for leave history
|   └── GeofenceEditorModal.tsx ← Popup to set geofence on map
|
├─ services/
|   ├── apiService.ts    ← All data fetching (demo JSON or real API)
|   └── exportService.ts ← CSV export logic
|
└─ data/
    ├── users.json       ← Demo user accounts
    ├── attendance.json  ← Demo check-in/out records
    └── leaves.json       ← Demo leave requests
```

Good luck tomorrow! You've got this. 🍀